PES-VCS Lab Report: Building a Version Control System

Student: PAVAN KISHOR M | **SRN:** PES2UG24AM111

Project Overview

This project involved building **PES-VCS**, a functional local version control system modeled after Git internals. The system implements content-addressable storage, directory sharding, staging (indexing), and commit history management through a Merkle Tree architecture.

Phase 1: Object Storage Foundation

In this phase, I implemented the core storage logic in `object.c`. This involved creating a content-addressable system where data is stored based on its SHA-256 hash. I implemented atomic writes by writing to temporary files before renaming them to their final sharded path.

Implementation Highlights

- **Header Construction**: Prepended type headers (blob, tree, or commit) to data before hashing.
- **Integrity Verification**: Recomputed hashes during read operations to ensure data hasn't been corrupted.
- **Sharding**: Objects are stored in subdirectories based on the first two characters of their hash to prevent directory overcrowding.

Screenshot 1.1: Test Objects Output

```
kish@debian:~/PES2UG24AM11-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
kish@debian:~/PES2UG24AM11-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Screenshot 1.2: Sharded Object Store

```
kish@debian:~/PES2UG24AM11-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Phase 2: Tree Objects

Phase 2 focused on representing directory structures. I implemented `tree_from_index` in `tree.c` using a recursive helper function to handle nested paths, allowing for a full snapshot of the project

hierarchy.

Implementation Highlights

- **Recursive Construction**: The system builds sub-tree objects for every directory level found in the index.
- **Serialization**: Tree entries are sorted by name and converted into a binary format containing file modes, names, and hashes.

Screenshot 2.1: Test Tree Output

```
kish@debian:~/PES2UG24AM11-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
kish@debian:~/PES2UG24AM11-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Screenshot 2.2: Raw Tree Object Inspection

```
kish@debian:~/PES2UG24AM11-pes-vcs$ xxd .pes/objects/dd/bf6fe947a94d94d02b3914f89d777ee93b815dbd0b39f4264d56452ce7d7c3
00000000: 636f 6d6d 6974 2032 3631 0074 7265 6520  commit 261.tree
00000010: 3936 6330 6435 6332 3864 6433 6237 3139  96c0d5c28dd3b719
00000020: 3630 3663 3336 3732 3363 6431 3661 6566  606c36723cd16aef
00000030: 6263 3733 6233 6463 3862 3734 6465 3631  bc73b3dc8b74de61
00000040: 3037 3031 3833 3335 3764 3639 3638 6239  07018335 7d6968b9
00000050: 0a70 6172 656e 7420 6331 3733 3461 6538  .parent c1734ae8
00000060: 3635 6239 6135 6138 3338 3930 3666 6533  65b9a5a838906fe3
00000070: 6435 3731 3964 3138 6431 3733 3837 3938  d5719d18d1738798
00000080: 6162 3739 3136 3432 6130 3737 6662 6530  ab791642a077fbe0
00000090: 6661 3361 6264 3465 0a61 7574 686f 7220  fa3abd4e.author
000000a0: 5461 6869 7220 5369 6464 6971 7565 203c  Tahir Siddique <
000000b0: 5045 5332 5547 3234 414d 3039 323e 2031  PES2UG24AM092> 1
000000c0: 3737 3634 3035 3732 370a 636f 6d6d 6974  776405727.commit
000000d0: 7465 7220 5461 6869 7220 5369 6464 6971  ter Tahir Siddiq
000000e0: 7565 203c 5045 5332 5547 3234 414d 3039  ue <PES2UG24AM09
000000f0: 323e 2031 3737 3634 3035 3732 370a 0a41  2> 1776405727..A
00000100: 6464 2066 6172 6577 656c 6c20 6669 6c65  dd farewell file
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Phase 3: The Index (Staging Area)

I implemented the staging area in `index.c`, which serves as the "middle ground" between the working directory and the permanent object store.

Implementation Highlights

- **Atomic Save**: The index is written to a temporary file and renamed only after a successful `fsync` to ensure durability.
- **Metadata Tracking**: The index stores file modes, SHA-256 hashes, modification times, and sizes for change detection.

Screenshot 3.1: Init, Add, and Status Sequence

```
kish@debian:~/PES2UG24AM11-pes-vcs$ make pes
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
kish@debian:~/PES2UG24AM11-pes-vcs$ echo "hello" > file1.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ echo "world" > file2.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes add file1.txt file2.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects
  untracked:  pes.h
  untracked:  PROGRESS.md
  untracked:  pes.c
  untracked:  object.c.save
  untracked:  tree.c
  untracked:  Makefile
  untracked:  object.c
  untracked:  test_tree
  untracked:  commit.h
  untracked:  README.md
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  index.h

kish@debian:~/PES2UG24AM11-pes-vcs$
```

Screenshot 3.2: Human-Readable Index

```
kish@debian:~/PES2UG24AM11-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776405271 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776405279 6 file2.txt
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Phase 4: Commits and History

The final implementation phase involved `commit.c`. I implemented `commit_create`, which snapshots the current index, links it to a parent commit, and updates the `HEAD` reference.

Implementation Highlights

- **Commit Chain**: Each commit points to its predecessor, creating a verifiable link of project history.
- **Author Metadata**: Integrated the `PES_AUTHOR` environment variable to record who made the changes.

Screenshot 4.1: Commit Log

```
kish@debian:~/PES2UG24AM11-pes-vcs$ make pes
make: 'pes' is up to date.
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
kish@debian:~/PES2UG24AM11-pes-vcs$ echo "Hello" > hello.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes add hello.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ echo "World" >> hello.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes add hello.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes commit -m "Add world to hello.txt"
Committed: b3a5caac90ff... Add world to hello.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ echo "Goodbye" > bye.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes add bye.txt
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes commit -m "Add farewell file"
Committed: 9cb4bc0ddaf4... Add farewell file
kish@debian:~/PES2UG24AM11-pes-vcs$ ./pes log
commit 9cb4bc0ddaf4f8bf7b8f9f112fc56d508d54ed6036b4ab6e87e161233dbccb25
Author: PAVAN KISHOR M <PES2UG24AM111>
Date:   1776405903

    Add farewell file

commit b3a5caac90ff65d675d140d159ed0f754bf01a0803ba256096bf40a488d1a98a
Author: PAVAN KISHOR M <PES2UG24AM111>
Date:   1776405883

    Add world to hello.txt
```

Screenshot 4.2: Object Store Growth

```
kish@debian:~/PES2UG24AM11-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/4960c6d5a08feb437e844aa4d2db4c9d23cf4260905c1dd335f6288ca27534
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719006c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/9c/b4bc0ddaf4f8bf7b8f9f112fc56d508d54ed6036b4ab6e87e161233dbccb25
.pes/objects/b3/a5caac90ff65d675d140d159ed0f754bf01a0803ba256096bf40a488d1a98a
.pes/objects/c1/734ae865b9a5a838906fe3d5719d18d1738798ab791642a077fbe0fa3abd4e
.pes/objects/dd/bf6fe947a94d94d02b3914f89d777ee93b815dbd0b39f4264d56452ce7d7c3
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Screenshot 4.3: Reference Chain

```
kish@debian:~/PES2UG24AM11-pes-vcs$ cat .pes/refs/heads/main
9cb4bc0ddaf4f8bf7b8f9f112fc56d508d54ed6036b4ab6e87e161233dbccb25
kish@debian:~/PES2UG24AM11-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
kish@debian:~/PES2UG24AM11-pes-vcs$
```

Phase 5 & 6: Analysis Answers

Q5.1 — Implementing `pes checkout <branch>`

To implement `checkout`:

- **Reference Update**: Update `.pes/HEAD` to point to the target branch (e.g., `ref: refs/heads/<branch>`).
- **Working Directory Restoration**: Retrieve the commit hash from the branch file, find its root tree, and recursively restore every blob to its respective file path in the working directory.
- **Complexity**: The operation is complex because it must check for uncommitted changes (dirty files) to prevent overwriting user work that isn't saved in a commit.

Q5.2 — Detecting Dirty Working Directory Conflicts

To detect a conflict:

1. For each tracked file, compare its on-disk metadata (mtime/size) to the index record to see if it is "dirty".
2. Look up that path in the target branch's tree.
3. If the blob hash in the target branch differs from the current index AND the file is dirty, refuse the checkout.

Q5.3 — Detached HEAD State

In a "Detached HEAD" state, `HEAD` points directly to a commit hash instead of a branch.

- **Behavior**: New commits form a valid chain, but no branch pointer (like `main`) is updated to follow them.
- **Recovery**: If a user switches branches, these commits become orphaned (unreachable). They can be recovered by finding the hash in `pes log` and creating a new branch pointing to that hash.

Q6.1 — Garbage Collection Algorithm

To reclaim space:

1. **Traverse**: Start from all known branch refs and walk every commit, tree, and blob.
2. **Mark**: Use a Hash Set to track all reachable ObjectIDs.
3. **Sweep**: Delete any file in `.pes/objects/` that is not in the reachable set.

- **Scale**: For 100,000 commits, this involves visiting roughly 50 million objects (assuming 500 files per snapshot), where a Hash Set ensures $O(1)$ lookups.

Q6.2 — GC Race Conditions

A race condition occurs if GC deletes an object that an in-progress commit is currently writing but hasn't yet linked into a commit object.

- **Git's Solution**: Git uses a "grace period" (typically 2 weeks), where objects younger than a certain threshold are never deleted by GC, even if they appear unreachable.

Final Integration

The entire suite was verified using the integration test script.

Final Screenshot: Integration Success

```
kish@debian:~/PES2UG24AM11-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 715fba3f4fbc... Initial commit

Log after first commit:
commit 715fba3f4fbc8f9159823693eada73ee405750ebba4fc3832a6367e29aea82a2
Author: PAVAN KISHOR M <PES2UG24AM111>
Date:   1776406183

    Initial commit


--- Second Commit ---
Committed: c8c09846ae96... Update file.txt

--- Third Commit ---
Committed: cea7e2d9c5c5... Add farewell

--- Full History ---
commit cea7e2d9c5c55ac89195c32abe4893c01a26642dabc7e2ecace18be13297bb93
Author: PAVAN KISHOR M <PES2UG24AM111>
Date:   1776406183

    Add farewell
```

```
commit c8c09846ae96f30ac4019367d2aeab3c6bda6806b0c42930ba5f8a54934021af
Author: PAVAN KISHOR M <PES2UG24AM111>
Date:   1776406183

    Update file.txt

commit 715fba3f4fbc8f9159823693eada73ee405750ebba4fc3832a6367e29aea82a2
Author: PAVAN KISHOR M <PES2UG24AM111>
Date:   1776406183

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
cea7e2d9c5c55ac89195c32abe4893c01a26642dabc7e2ecace18be13297bb93

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/71/5fba3f4fbc8f9159823693eada73ee405750ebba4fc3832a6367e29aea82a2
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c8/c09846ae96f30ac4019367d2aeab3c6bda6806b0c42930ba5f8a54934021af
.pes/objects/ce/a7e2d9c5c55ac89195c32abe4893c01a26642dabc7e2ecace18be13297bb93
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
kish@debian:~/PES2UG24AM11-pes-vcs$
```